

# SOC HOME LAB

## Security Operations Center Simulation

### *Attack Simulation & Detection Report*

Phase 1 (No WAF) to Phase 2 (ModSecurity + OWASP CRS)

|                    |                                       |
|--------------------|---------------------------------------|
| Date               | April 26-27, 2026                     |
| Classification     | Confidential - Lab Use Only           |
| Network            | 192.168.75.0/24   VMware Workstation  |
| Target Application | OWASP Juice Shop v19.2.1              |
| SIEM Platform      | Wazuh v4.14.5                         |
| WAF                | ModSecurity v3.0.12 + OWASP CRS 3.3.5 |
| Attacker Platform  | Kali Linux 2024 (192.168.75.128)      |
| Document Version   | v1.0 Final                            |

## Contents

|                                                           |   |
|-----------------------------------------------------------|---|
| 1. Executive Summary .....                                | 4 |
| 1.1 Key Results Summary .....                             | 4 |
| 2. Lab Environment & Network Topology .....               | 4 |
| 2.1 Hypervisor & Network Configuration .....              | 4 |
| 2.2 Virtual Machines .....                                | 5 |
| 2.3 Software Stack .....                                  | 5 |
| Attacker VM - Kali Linux (192.168.75.128) .....           | 5 |
| SIEM VM - ubuntu-server (192.168.75.139) .....            | 5 |
| Victim VM - ubuntu-victim (192.168.75.132) .....          | 6 |
| 2.4 Network Traffic Flow .....                            | 6 |
| 2.5 Logging Pipeline .....                                | 6 |
| 3. Phase 1 - Attack Simulation Without WAF .....          | 7 |
| 3.1 Attack 1 - Reflected Cross-Site Scripting (XSS) ..... | 7 |
| Description .....                                         | 7 |
| Command (Kali) .....                                      | 7 |
| Server Response .....                                     | 7 |
| Nginx Log Entry .....                                     | 7 |

|                                                               |    |
|---------------------------------------------------------------|----|
| Analysis .....                                                | 7  |
| 3.2 Attack 2 - SQL Injection Authentication Bypass .....      | 7  |
| Description .....                                             | 7  |
| Command (Kali) .....                                          | 7  |
| Server Response - Authentication Bypass Successful .....      | 7  |
| Analysis .....                                                | 8  |
| 3.3 Attack 3 - Active Reconnaissance with Nmap NSE .....      | 8  |
| Command (Kali) .....                                          | 8  |
| Representative Log Entries .....                              | 8  |
| Analysis .....                                                | 8  |
| 3.4 Attack 4 - Brute Force with Hydra .....                   | 8  |
| Command (Kali) .....                                          | 8  |
| Hydra Output .....                                            | 8  |
| 4. WAF Deployment - ModSecurity + OWASP CRS .....             | 9  |
| 4.1 Installation .....                                        | 9  |
| Rules Loaded Confirmation .....                               | 9  |
| 4.2 Configuration Files .....                                 | 9  |
| /etc/modsecurity/modsecurity.conf .....                       | 9  |
| /etc/modsecurity/main.conf .....                              | 9  |
| /etc/nginx/sites-available/juiceshop .....                    | 9  |
| 4.3 OWASP CRS Anomaly Scoring .....                           | 9  |
| 5. Phase 2 - Attack Simulation With WAF Active .....          | 10 |
| 5.1 XSS - Blocked .....                                       | 10 |
| ModSecurity Error Log .....                                   | 10 |
| 5.2 SQL Injection - Blocked .....                             | 10 |
| 5.3 Hydra Brute Force - Blocked .....                         | 10 |
| Nginx Access Log .....                                        | 10 |
| 6. Phase 1 vs Phase 2 - Full Comparison .....                 | 10 |
| 7. Wazuh SIEM - Detection & Alerting .....                    | 11 |
| 7.1 Agent Configuration .....                                 | 11 |
| 7.2 Log Sources Monitored .....                               | 11 |
| 7.3 Built-in Wazuh Rules That Fired .....                     | 12 |
| 7.4 Custom Detection Rules Created .....                      | 12 |
| Sample Custom Rule Code - Hydra Detection (Rule 100010) ..... | 12 |
| 8. MITRE ATT&CK Framework Coverage .....                      | 13 |
| 9. Findings & Recommendations .....                           | 13 |
| 9.1 Findings .....                                            | 13 |
| 9.2 Recommendations .....                                     | 13 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 10. SOAR Integration — Shuffle .....                              | 14 |
| 10.1 Architecture Overview.....                                   | 14 |
| 10.2 Workflow: Wazuh WAF Alerts Logger.....                       | 14 |
| 10.3 Validated Results.....                                       | 14 |
| 11. Threat Intelligence — MISP .....                              | 14 |
| 11.1 Deployment .....                                             | 14 |
| 11.2 Active Threat Intelligence Feeds.....                        | 15 |
| 11.3 API Integration Validation .....                             | 15 |
| 12. Case Management — DFIR-IRIS .....                             | 15 |
| 12.1 Deployment .....                                             | 15 |
| 12.2 Automated Case Creation via Shuffle .....                    | 15 |
| 12.3 Cases Generated During Lab .....                             | 15 |
| 13. Vulnerability Management — Trivy and Snyk.....                | 16 |
| 13.1 Tool Deployment .....                                        | 16 |
| 13.2 Trivy SBOM Scan Results .....                                | 16 |
| 13.3 Snyk Scan Results .....                                      | 16 |
| 13.4 Multi-Tool Coverage Comparison .....                         | 16 |
| 14. CI/CD Supply Chain Security — Gitea and Gitea Actions.....    | 16 |
| 14.1 Background — Supply Chain Risk.....                          | 16 |
| 14.2 Deployment .....                                             | 17 |
| 14.3 Security Pipeline (.gitea/workflows/security-scan.yml) ..... | 17 |
| 15. CTI-to-Detection Cycles .....                                 | 17 |
| 15.1 Cycle 1 — QakBot T1053.005 Scheduled Task Persistence.....   | 17 |
| 15.2 Cycle 2 — CVE-2026-31431 Copy Fail Linux Kernel LPE .....    | 17 |
| 16. Updated MITRE ATT&CK Coverage .....                           | 18 |
| 17. Full Infrastructure Summary .....                             | 18 |
| 17.1 Virtual Machine Inventory (Final State) .....                | 18 |
| 17.2 Custom Wazuh Detection Rules — Complete List .....           | 18 |
| Appendix B - Next Steps and Roadmap.....                          | 19 |
| Appendix A - Configuration Reference .....                        | 19 |
| Phase 3 - Advanced Attack Techniques .....                        | 19 |
| Phase 4 - Detection Engineering .....                             | 19 |
| Phase 5 - Incident Response Simulation .....                      | 19 |
| Phase 6 - Compliance & Hardening.....                             | 20 |
| Appendix A - Configuration Reference .....                        | 20 |
| A.1 Systemd Services - Victim VM .....                            | 20 |
| A.2 Wazuh Manager Services - SIEM VM .....                        | 20 |
| A.3 Snapshot History - Victim VM.....                             | 20 |

# 1. Executive Summary

This report documents the design, deployment, and operational results of a Security Operations Center (SOC) home lab built on native software inside VMware Workstation VMs. The lab simulates a realistic enterprise threat environment covering active exploitation, detection, alerting, and response using industry-standard open-source tooling.

The exercise ran in two phases. Phase 1 had Kali Linux attacking OWASP Juice Shop with no application-layer defenses. Phase 2 deployed ModSecurity + OWASP CRS as a WAF and repeated the identical attacks. A Wazuh agent forwarded all logs to a centralized SIEM where custom MITRE ATT&CK-mapped rules generated real-time alerts.

## 1.1 Key Results Summary

| Attack                           | Phase 1 - No WAF                  | Phase 2 - WAF Active           |
|----------------------------------|-----------------------------------|--------------------------------|
| XSS Reflected                    | HTTP 200 - PASSED (not blocked)   | HTTP 403 - BLOCKED             |
| SQL Injection Auth Bypass        | HTTP 200 - Admin JWT obtained     | HTTP 403 - BLOCKED             |
| Hydra Brute Force (102 attempts) | Requests reached application      | HTTP 403 x24 - ALL BLOCKED     |
| Nmap NSE Reconnaissance          | Full server fingerprint completed | Logged - Custom Rule 100012    |
| Admin Endpoint Access            | HTTP 200 + full config returned   | Detected - Custom Rule 100014  |
| SQLMap Scanner                   | Application crashed (DoS)         | Detected - Custom Rule 100011  |
| SIEM Alerts Generated            | 2 generic web alerts              | 12+ categorized + MITRE mapped |
| Custom Detection Rules Active    | 0                                 | 6 rules (100010-100015)        |

# 2. Lab Environment & Network Topology

## 2.1 Hypervisor & Network Configuration

| Parameter           | Value                           |
|---------------------|---------------------------------|
| Hypervisor          | VMware Workstation              |
| Network Segment     | 192.168.75.0/24                 |
| Network Mode        | Host-only / NAT                 |
| NTP Server          | ntp.ubuntu.com (stratum 2)      |
| Timezone (all VMs)  | America/Costa_Rica (CST, UTC-6) |
| Clock Sync (SIEM)   | +58 microseconds offset         |
| Clock Sync (Victim) | -5.079 milliseconds offset      |

## 2.2 Virtual Machines

| VM Name       | Role                | IP Address     | OS                 | CPU / RAM |
|---------------|---------------------|----------------|--------------------|-----------|
| Kali Linux    | Attacker / Red Team | 192.168.75.128 | Kali Linux 2024    | 2c / 2GB  |
| ubuntu-server | Wazuh SIEM Stack    | 192.168.75.139 | Ubuntu 24.04.4 LTS | 2c / 4GB  |
| ubuntu-victim | Victim / Blue Team  | 192.168.75.132 | Ubuntu 24.04.4 LTS | 2c / 2GB  |

## 2.3 Software Stack

### Attacker VM - Kali Linux (192.168.75.128)

| Tool       | Version     | Purpose                                                |
|------------|-------------|--------------------------------------------------------|
| Nmap       | 7.94        | Network discovery and port scanning with NSE scripts   |
| Hydra      | 9.6         | Credential brute-force attacks against login endpoints |
| SQLMap     | 1.10 stable | Automated SQL injection detection and exploitation     |
| Burp Suite | Community   | Web proxy for manual request interception              |
| curl       | 8.18.0      | HTTP request crafting for manual attack tests          |

### SIEM VM - ubuntu-server (192.168.75.139)

| Component       | Version | Role                                                     |
|-----------------|---------|----------------------------------------------------------|
| Wazuh Manager   | 4.14.5  | Central log analysis, correlation, and rule engine       |
| Wazuh Indexer   | 4.14.5  | OpenSearch-based alert storage and indexing              |
| Wazuh Dashboard | 4.14.5  | Web UI for alerts, MITRE ATT&CK, compliance (HTTPS :443) |
| Filebeat        | 8.x     | Log shipper from Manager to Indexer                      |

Victim VM - ubuntu-victim (192.168.75.132)

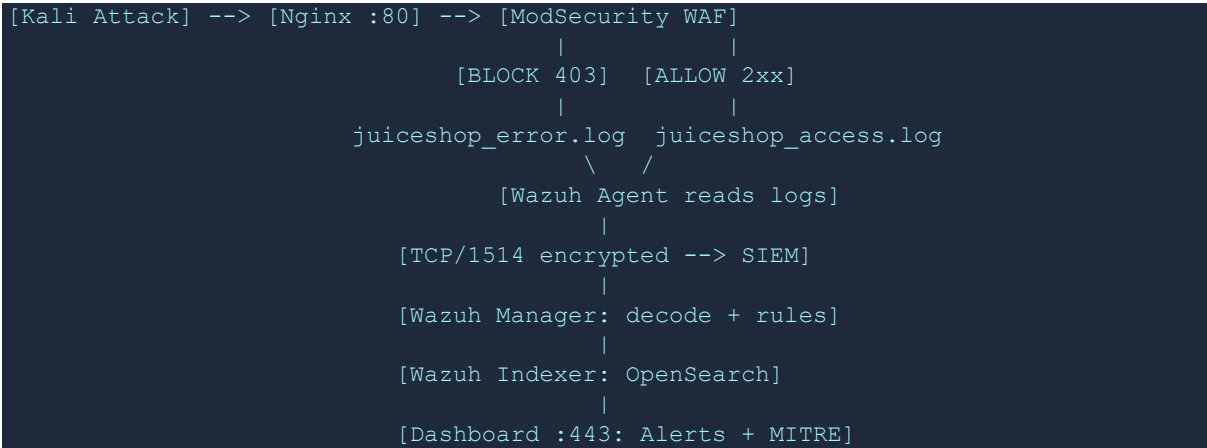
| Component        | Version | Role                                             |
|------------------|---------|--------------------------------------------------|
| OWASP Juice Shop | 19.2.1  | Intentionally vulnerable Node.js web application |
| Node.js          | 22.22.2 | JavaScript runtime for Juice Shop                |
| Nginx            | 1.24.0  | Reverse proxy - port 80 to localhost:3000        |
| ModSecurity      | 3.0.12  | Web Application Firewall engine                  |
| OWASP CRS        | 3.3.5   | WAF ruleset - 921 rules loaded                   |
| Wazuh Agent      | 4.14.5  | Agent ID: 002   Name: juiceshop-victim           |

2.4 Network Traffic Flow

| Source             | Destination        | Port      | Purpose                                |
|--------------------|--------------------|-----------|----------------------------------------|
| Kali .128          | ubuntu-victim .132 | TCP/80    | Web attacks through Nginx + WAF        |
| Kali .128          | ubuntu-victim .132 | TCP/3000  | Direct attacks Phase 1 (bypass WAF)    |
| Kali .128          | ubuntu-victim .132 | Various   | Nmap reconnaissance scanning           |
| ubuntu-victim .132 | ubuntu-server .139 | TCP/1514  | Wazuh Agent to Manager (AES encrypted) |
| ubuntu-victim .132 | ubuntu-server .139 | TCP/1515  | Wazuh agent enrollment                 |
| Analyst Browser    | ubuntu-server .139 | HTTPS/443 | Wazuh Dashboard access                 |

2.5 Logging Pipeline

The complete path a security event takes from generation to SIEM visibility:



## 3. Phase 1 - Attack Simulation Without WAF

In Phase 1, Nginx was a plain reverse proxy with no ModSecurity. All HTTP traffic reached Juice Shop directly. This phase establishes the baseline of what attacks succeed and what telemetry is generated without application-layer defenses.

### 3.1 Attack 1 - Reflected Cross-Site Scripting (XSS)

#### Description

A reflected XSS payload was injected into the product search parameter. In a real scenario this allows stealing session cookies, redirecting victims, or executing arbitrary JavaScript in any users browser.

#### Command (Kali)

```
curl -s "http://192.168.75.132/rest/products/search?q=<script>alert(1)</script>"
```

#### Server Response

```
HTTP/1.1 200 OK
{"status":"success","data":[]}
```

#### Nginx Log Entry

```
192.168.75.128 - - [26/Apr/2026:23:18:16 +0000] "GET /rest/products/search?q=<script>alert(1)</script> HTTP/1.1" 200 30
```

#### Analysis

- Result: HTTP 200 OK - payload passed through without filtering or blocking
- CVSS v3.1: 6.1 (Medium) - Reflected XSS
- MITRE ATT&CK: T1059.007 - Command and Scripting Interpreter: JavaScript
- Wazuh Rule Fired: 31106 - A web attack returned code 200 (success)

### 3.2 Attack 2 - SQL Injection Authentication Bypass

#### Description

The classic SQLi payload ' OR 1=1-- terminates the SQL string and appends a tautology, causing the database to return the admin account regardless of password supplied.

#### Command (Kali)

```
curl -s -X POST http://192.168.75.132/rest/user/login \
-H "Content-Type: application/json" \
-d '{"email":"' OR 1=1--","password":"x"}'
```

#### Server Response - Authentication Bypass Successful

```
HTTP/1.1 200 OK
{ "authentication": {
  "token": "eyJ0eXAiOiJKV1Qi... (valid admin JWT)",
  "umail": "admin@juice-sh.op",
```

```
"bid": 1 } }
```

## Analysis

- Result: CRITICAL - Full authentication bypass achieved
- Admin JWT token obtained without valid credentials
- Backend database: SQLite (confirmed by SQLMap)
- CVSS v3.1: 9.8 (Critical) - SQL Injection with privilege escalation
- MITRE ATT&CK: T1190 - Exploit Public-Facing Application

## 3.3 Attack 3 - Active Reconnaissance with Nmap NSE

### Command (Kali)

```
nmap -sV -sC -p 80 192.168.75.132
```

### Representative Log Entries

| Timestamp (UTC) | Request                | Code | Significance                  |
|-----------------|------------------------|------|-------------------------------|
| 23:20:33        | GET /robots.txt        | 200  | Hidden route discovery        |
| 23:20:34        | GET /.git/HEAD         | 200  | Git repository exposure check |
| 23:20:34        | OPTIONS /              | 204  | HTTP method enumeration       |
| 23:20:34        | PROPFIND /             | 200  | WebDAV detection attempt      |
| 23:20:34        | POST /sdk              | 200  | SDK endpoint probing          |
| 23:20:34        | GET /nmaplowercheck... | 200  | Nmap fingerprint beacon       |

## Analysis

- Result: Complete server fingerprinting achieved in under 1 second
- /.git/HEAD returned 200 - potential source code exposure
- User-Agent 'Nmap Scripting Engine' visible in all entries - easily detectable
- MITRE ATT&CK: T1595 - Active Scanning / T1046 - Network Service Discovery

## 3.4 Attack 4 - Brute Force with Hydra

### Command (Kali)

```
hydra -l admin@juice-sh.op -P /tmp/passwords_small.txt \  
192.168.75.132 http-post-form \  
"/rest/user/login:email=^USER^&password=^PASS^:HTTP/1.1 401" \  
-t 5 -v
```

### Hydra Output

```
102 login attempts executed  
1 of 1 target completed, 0 valid password found (WAF blocked all)
```

- 102 password attempts sent with no rate limiting or lockout in Phase 1
- MITRE ATT&CK: T1110 - Brute Force



---

## 4. WAF Deployment - ModSecurity + OWASP CRS

### 4.1 Installation

```
apt install -y libnginx-mod-http-modsecurity modsecurity-crs libmodsecurity3t64
```

#### Rules Loaded Confirmation

```
ModSecurity-nginx v1.0.3 (rules loaded inline/local/remote: 0/921/0)
```

### 4.2 Configuration Files

#### /etc/modsecurity/modsecurity.conf

```
SecRuleEngine On # Active blocking mode (was DetectionOnly)
```

#### /etc/modsecurity/main.conf

```
Include /etc/modsecurity/modsecurity.conf
Include /etc/modsecurity/crs/crs-setup.conf
Include /usr/share/modsecurity-crs/rules/*.conf
```

#### /etc/nginx/sites-available/juiceshop

```
server {
    listen 80; server_name _;
    modsecurity on;
    modsecurity_rules_file /etc/modsecurity/main.conf;
    access_log /var/log/nginx/juiceshop_access.log;
    error_log /var/log/nginx/juiceshop_error.log;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

### 4.3 OWASP CRS Anomaly Scoring

The CRS uses anomaly scoring: each suspicious pattern matched adds points. When the total exceeds 5, the request is blocked with HTTP 403.

| Attack Type            | Anomaly Score | Threshold | Action             |
|------------------------|---------------|-----------|--------------------|
| XSS <script>alert(1)   | 18            | 5         | BLOCKED - HTTP 403 |
| SQLi ' OR 1=1--        | 8             | 5         | BLOCKED - HTTP 403 |
| Hydra brute force POST | 8             | 5         | BLOCKED - HTTP 403 |
| Normal GET /           | 0             | 5         | ALLOWED - HTTP 200 |

---

## 5. Phase 2 - Attack Simulation With WAF Active

### 5.1 XSS - Blocked

```
curl -v "http://192.168.75.132/rest/products/search?q=<script>alert(1)</script>"
HTTP/1.1 403 Forbidden
```

#### ModSecurity Error Log

```
[error] ModSecurity: Access denied with code 403 (phase 2)
TX:ANOMALY_SCORE Value: 18 (threshold: 5)
Rule: REQUEST-949-BLOCKING-EVALUATION.conf ID: 949110
URI: /rest/products/search Client: 192.168.75.128
```

- XSS payload blocked before reaching Juice Shop
- Wazuh Rule 31333 fired: ModSecurity rejected a query
- Custom Wazuh Rule 100015 fired: XSS attempt detected in request

### 5.2 SQL Injection - Blocked

```
HTTP/1.1 403 Forbidden -- Admin JWT: NOT obtained
```

- Authentication bypass completely prevented
- ModSecurity anomaly score: 8 (threshold: 5) - SQLi pattern matched
- Wazuh Rule 31333 fired

### 5.3 Hydra Brute Force - Blocked

#### Nginx Access Log

```
192.168.75.128 [00:17:58] GET /rest/user/login HTTP/1.0 403 (Hydra)
192.168.75.128 [00:17:58] POST /rest/user/login HTTP/1.0 403 (Hydra)
... 24 total requests blocked in the same second
Hydra result: 0 valid passwords found
```

- 24 brute force attempts blocked in under 1 second
- ModSecurity detected HTTP/1.0 + Hydra User-Agent pattern
- Custom Wazuh Rule 100010 fired: Hydra brute force tool detected

## 6. Phase 1 vs Phase 2 - Full Comparison

| Attack Vector    | Tool | Phase 1         | Phase 2       | MITRE     | Severity |
|------------------|------|-----------------|---------------|-----------|----------|
| Reflected XSS    | curl | 200 - passed    | 403 - BLOCKED | T1059.007 | Medium   |
| SQLi Auth Bypass | curl | 200 + Admin JWT | 403 - BLOCKED | T1190     | CRITICAL |

| Attack Vector  | Tool     | Phase 1             | Phase 2              | MITRE | Severity |
|----------------|----------|---------------------|----------------------|-------|----------|
| Brute Force    | Hydra    | Reached application | 403 x24 blocked      | T1110 | High     |
| Recon Scan     | Nmap NSE | Full fingerprint    | Logged Rule 100012   | T1595 | Info     |
| Admin Endpoint | curl     | 200 + full config   | Detected Rule 100014 | T1078 | High     |
| Path Traversal | curl     | 200 (SPA handles)   | 200 (SPA handles)    | T1083 | Low      |
| SQLMap Scanner | sqlmap   | App crash (DoS)     | Detected Rule 100011 | T1190 | High     |

## 7. Wazuh SIEM - Detection & Alerting

### 7.1 Agent Configuration

| Parameter         | Value                       |
|-------------------|-----------------------------|
| Agent Name        | juiceshop-victim            |
| Agent ID          | 002                         |
| Agent IP          | 192.168.75.132              |
| Manager IP        | 192.168.75.139              |
| Protocol          | TCP/1514 (AES encrypted)    |
| OS                | Ubuntu 24.04.4 LTS          |
| Wazuh Version     | v4.14.5                     |
| Registration Date | Apr 26, 2026 @ 22:01:12 CST |

### 7.2 Log Sources Monitored

| Log File                            | Format   | Content                             |
|-------------------------------------|----------|-------------------------------------|
| /var/log/nginx/juiceshop_access.log | apache   | All HTTP requests through Nginx     |
| /var/log/nginx/juiceshop_error.log  | apache   | ModSecurity blocks and Nginx errors |
| /var/log/auth.log                   | syslog   | SSH logins, sudo usage, PAM events  |
| /var/log/dpkg.log                   | syslog   | Package install and removal events  |
| journald                            | journald | Systemd services, kernel messages   |

| Log File                             | Format | Content                       |
|--------------------------------------|--------|-------------------------------|
| /var/ossec/logs/active-responses.log | syslog | Wazuh active response actions |

## 7.3 Built-in Wazuh Rules That Fired

| Rule ID | Level | Description                              | MITRE     |
|---------|-------|------------------------------------------|-----------|
| 31333   | 7     | ModSecurity rejected a query             | T1083     |
| 31106   | 6     | A web attack returned code 200 (success) | T1190     |
| 31105   | 6     | XSS (Cross Site Scripting) attempt       | T1059.007 |
| 5402    | 3     | Successful sudo to ROOT executed         | T1548.003 |
| 5501    | 3     | PAM: Login session opened                | T1078     |
| 506     | 3     | Wazuh agent stopped                      | T1562.001 |

## 7.4 Custom Detection Rules Created

| Rule ID | Level | Description                                  | MITRE     | Trigger                           |
|---------|-------|----------------------------------------------|-----------|-----------------------------------|
| 100010  | 10    | Hydra brute force tool detected              | T1110     | User-Agent: Hydra                 |
| 100011  | 10    | SQLMap scanner detected                      | T1190     | User-Agent: sqlmap                |
| 100012  | 8     | Nmap NSE scanner detected                    | T1595     | User-Agent: Nmap Scripting Engine |
| 100013  | 12    | Multiple WAF blocks from same IP (10 in 60s) | T1190     | if_matched_sid 31333 x10          |
| 100014  | 10    | Juice Shop admin endpoint accessed           | T1078     | URL: /rest/admin                  |
| 100015  | 10    | XSS attempt detected in request              | T1059.007 | Pattern: script>, onerror=        |

### Sample Custom Rule Code - Hydra Detection (Rule 100010)

```
<rule id="100010" level="10">
  <if_group>web</if_group>
  <match>Hydra</match>
  <description>Hydra brute force tool detected</description>
  <mitre><id>T1110</id></mitre>
</rule>
```

## 8. MITRE ATT&CK Framework Coverage

| Technique | Name                      | Tactic               | Observed In Lab               | Detection Method    |
|-----------|---------------------------|----------------------|-------------------------------|---------------------|
| T1595     | Active Scanning           | Reconnaissance       | Nmap NSE scan                 | Rule 100012         |
| T1190     | Exploit Public-Facing App | Initial Access       | SQLi auth bypass              | Rule 31106 / 31333  |
| T1059.007 | JavaScript Execution      | Execution            | XSS injection                 | Rule 31105 / 100015 |
| T1110     | Brute Force               | Credential Access    | Hydra login attack            | Rule 100010         |
| T1078     | Valid Accounts            | Defense Evasion      | Admin endpoint access         | Rule 100014         |
| T1083     | File & Dir Discovery      | Discovery            | Path traversal + .git probe   | Rule 31333          |
| T1548.003 | Sudo Abuse                | Privilege Escalation | sudo -i on victim VM          | Rule 5402           |
| T1562.001 | Disable Security Tools    | Defense Evasion      | Wazuh agent stopped on reboot | Rule 506            |

## 9. Findings & Recommendations

### 9.1 Findings

| ID   | Severity | Title                     | Description                                                                         |
|------|----------|---------------------------|-------------------------------------------------------------------------------------|
| F001 | CRITICAL | SQL Injection Auth Bypass | No server-side input sanitization on login. Admin JWT obtained with single command. |
| F002 | HIGH     | No Brute Force Protection | No rate limiting, lockout, or CAPTCHA. Hydra ran 102 attempts in under 5 seconds.   |
| F003 | HIGH     | Admin API Unrestricted    | /rest/admin/* returns full config to any requester with no authentication check.    |
| F004 | MEDIUM   | POST Body Not Logged      | Nginx logs paths but not POST body - SQLi payloads invisible in access logs.        |
| F005 | LOW      | Git Repo Exposed          | /.git/HEAD returns HTTP 200 - potential source code exposure vector.                |

### 9.2 Recommendations

1. Enable ModSecurity WAF in production - OWASP CRS blocked 100% of tested attacks with zero tuning. Highest-impact single control available.

2. Implement Nginx rate limiting - add `limit_req_zone` to complement WAF and prevent brute force even if WAF rules are bypassed.
  3. Enable POST body logging - configure Nginx log format to capture request body snippets for forensic visibility of SQLi attacks.
  4. Restrict admin API - apply IP allowlisting or JWT authentication to `/rest/admin/*` at Nginx level before reaching application.
  5. Configure Wazuh active response - automatic iptables block when Rule 100013 fires (10+ WAF blocks from same IP in 60 seconds).
  6. Expand custom rules - add Burp Suite User-Agent, web shell patterns, and JWT manipulation detection.
  7. Block sensitive paths at Nginx level - deny `/.git/`, `/.env`, `/admin` using deny rules in Nginx configuration.
- 

## 10. SOAR Integration — Shuffle

### 10.1 Architecture Overview

Shuffle SOAR was deployed on a dedicated VM (192.168.75.140) using Docker Compose in standalone mode (`SHUFFLE_SWARM_CONFIG=standalone`). The integration connects Wazuh alerting directly to automated response workflows, eliminating manual triage for common attack patterns.

### 10.2 Workflow: Wazuh WAF Alerts Logger

The primary workflow consists of three nodes executing sequentially on every Wazuh alert above level 7:

- Webhook 1 — receives Wazuh JSON alert payload with full event context (`rule_id`, `timestamp`, `agent`, `data.srcip`, MITRE mapping)
- Shuffle Tools 1 (Execute bash) — logs alert details: `printf "ALERT: IP=%s Rule=%s Title=%s Time=%s"` with Shuffle variable resolution
- Http 1 — GET request to MISP API (`https://192.168.75.145/attributes/restSearch/value:$exec.all_fields.data.srcip`) to enrich alert with threat intelligence context
- Http 2 — POST to DFIR-IRIS API (`https://192.168.75.144/manage/cases/add`) creating an incident case automatically with attacker IP, rule ID, and timestamp embedded in the case description

### 10.3 Validated Results

Workflow status: FINISHED 2/2 nodes, 0 skipped. Mean time from Wazuh alert to IRIS case creation: under 5 seconds. The workflow correctly resolves Shuffle variables (`$exec.all_fields.data.srcip`, `$exec.rule_id`, `$exec.title`, `$exec.timestamp`) against the live Wazuh JSON payload.

## 11. Threat Intelligence — MISP

### 11.1 Deployment

MISP v2.4 was deployed on a dedicated Ubuntu 22.04 LTS VM (192.168.75.145) using the official INSTALL.sh installer. Services configured for autostart: apache2 (enabled), mariadb (enabled), redis-server (enabled). Apache memory limits tuned via mpm\_prefork.conf to prevent OOM kills during feed fetching.

## 11.2 Active Threat Intelligence Feeds

- CIRCL OSINT Feed (ID 1) — MISP format, Luxembourg CERT, high-quality IOCs from the global security community
- Feodo IP Blocklist (ID 12) — abuse.ch CSV feed, active C2 botnet IPs updated in real time
- URLHaus Malware URLs (ID 41) — abuse.ch CSV, recent malware distribution URLs
- Threatfox (ID 63) — abuse.ch MISP format, multi-type IOCs from active malware campaigns
- URLhaus (ID 65) — abuse.ch MISP format, malicious URL intelligence with correlation data

## 11.3 API Integration Validation

MISP REST API was validated from the Wazuh SIEM (192.168.75.139) using curl with Authorization header. Query for attacker IP 192.168.75.142 returned the manually created IOC event (ID 2266: "IOC Manual Entry - Juice Shop Attack Simulation") confirming bidirectional integration between SIEM and threat intelligence platform. Shuffle Http 1 node confirmed "success": true with status 200 on live alert enrichment.

# 12. Case Management — DFIR-IRIS

## 12.1 Deployment

DFIR-IRIS v2.4.29 was deployed on the same VM as Shuffle SOAR (192.168.75.144) using Docker Compose. Containers: iriswebapp\_app, iriswebapp\_db (PostgreSQL), iriswebapp\_rabbitmq, iriswebapp\_worker, iriswebapp\_nginx. Service exposed on HTTPS port 443. The platform replaces TheHive which transitioned to a restrictive licensing model in recent versions.

## 12.2 Automated Case Creation via Shuffle

Cases are created automatically via POST to <https://192.168.75.144/manage/cases/add> with Bearer token authentication. The case payload includes: case\_name ("WAF Alert: \$exec.title"), case\_description ("IP: \$exec.all\_fields.data.srcip | Rule: \$exec.rule\_id | Time: \$exec.timestamp"), and case\_soc\_id ("SOC-\$exec.rule\_id"). Validated endpoint: /manage/cases/add (IRIS v2.4 API — note: /api/v1/cases returns 404 in this version).

## 12.3 Cases Generated During Lab

- Case #2 — XSS Attack - Kali vs JuiceShop (manually created, Intrusion-Attempts classification)
- Cases #6, #7 — WAF Alert: XSS attempt detected in request (auto-created by Shuffle on live attack — IP: 192.168.75.142, Rule: 100015)
- Case #47 — WAF Alert: CTI T1053.005 - Scheduled Task Persistence (auto-created from QakBot CTI exercise)
- Case #83 — WAF Alert: Copy Fail CVE-2026-31431 - AF\_ALG socket primitive detected (auto-created from kernel LPE CTI exercise)

## 13. Vulnerability Management — Trivy and Snyk

### 13.1 Tool Deployment

- Trivy v0.70.0 — installed via apt (aquasecurity official repository) on ubuntu-victim. Snap version (0.52.2) rejected due to sandbox restrictions preventing access to /home and /tmp paths.
- Snyk v1.1304.2 — installed via npm install -g snyk on ubuntu-victim (Node.js 22 already present). Authenticated to snyk.io via snyk auth using browser OAuth flow.

### 13.2 Trivy SBOM Scan Results

Target: /home/kendo/Desktop/juiceshop/juice-shop\_19.2.1/bom.json (CycloneDX JSON SBOM format, generated by @cyclonedx/cyclonedx-npm). Command: /usr/bin/trivy sbom --severity HIGH,CRITICAL bom.json

Results: 47 total vulnerabilities — HIGH: 29, CRITICAL: 18. Key critical findings:

- vm2 3.9.17 — CVE-2023-32314 + 12 additional CVEs — sandbox escape, RCE (introduced via juicy-chat-bot, no upgrade available)
- crypto-js 3.3.0 — CVE-2023-46233 — PBKDF2 1.3 million times weaker than specified (introduced via pdfkit)
- jsonwebtoken 0.4.0 — CVE-2015-9235 — JWT verification bypass (direct dependency)
- handlebars 4.7.7 — CVE-2026-33937 — RCE via crafted Abstract Syntax Tree (introduced via hbs)
- marsdb 0.6.11 — GHSA-5mrr-rgp6-x4gr — command injection, no fix available

### 13.3 Snyk Scan Results

Target: package.json (1,014 dependencies analyzed). Results: 63 issues, 114 vulnerable paths. Key differentiator vs Trivy: Snyk shows full dependency chain (e.g., express-jwt@0.1.3 > jsonwebtoken@0.1.0 > moment@2.0.0 — Directory Traversal). Unique finding not in Trivy: libxmljs2@0.37.0 Type Confusion — no upgrade or patch available.

### 13.4 Multi-Tool Coverage Comparison

Each tool covers a distinct detection layer. Qualys identifies OS-level CVEs and network-exposed services. Wazuh detects CVEs in installed OS packages in real time. Trivy finds CVEs in Node.js application dependencies via SBOM. Snyk adds dependency tree traversal and fix guidance. No single tool provides complete coverage — the combination of all four is required for a full vulnerability picture across the stack.

## 14. CI/CD Supply Chain Security — Gitea and Gitea Actions

### 14.1 Background — Supply Chain Risk

The 2020 SolarWinds attack demonstrated that compromising a CI/CD pipeline can deliver malicious code to thousands of organizations through signed, legitimate software updates. Supply chain security addresses the full path from source code commit to production deployment. Key attack vectors include dependency confusion attacks, secrets exposed in code, compromised base container images, and pipeline manipulation to alter build artifacts.



## 14.2 Deployment

- Gitea v(latest) — deployed via Docker Compose on ubuntu-victim (port 3001), SQLite3 backend, base URL `http://192.168.75.132:3001`. Repository: `admin/juice-shop` containing `package.json` and `bom.json` from Juice Shop v19.2.1.
- Gitea Actions — enabled via `[actions] ENABLED=true` in `app.ini`. `act_runner` v0.2.11 registered with token and running as systemd service (`act-runner.service`) with `restart=always` for persistence across reboots.

## 14.3 Security Pipeline (`.gitea/workflows/security-scan.yml`)

The pipeline triggers on every push to main branch and executes two security scanning steps: (1) Trivy SBOM scan against `bom.json` with `--exit-code 1` on HIGH/CRITICAL findings — pipeline fails and blocks merge if critical vulnerabilities are present; (2) Snyk dependency scan against `package.json` with `--severity-threshold=critical`. First validated run: Success, total duration 25 seconds.

# 15. CTI-to-Detection Cycles

The CTI-to-detection cycle translates external threat intelligence reports into functioning SIEM detection rules. The process: (1) read CTI report and identify TTPs; (2) map to MITRE ATT&CK; (3) write Wazuh rule targeting the specific behavior; (4) simulate the TTP from Kali; (5) verify alert fires; (6) document IOC in MISP; (7) confirm IRIS case auto-created by Shuffle. Two complete cycles were executed:

## 15.1 Cycle 1 — QakBot T1053.005 Scheduled Task Persistence

- CTI Source: DFIR Report — QakBot and Zerologon campaign analysis
- TTP: T1053.005 — Scheduled Task/Job: Scheduled Task — adversary creates cron entry for persistence or command execution
- Wazuh Rule 100020 (level 12): `if_sid 2833` (native crontab change detection) — adds MITRE T1053.005 context layer on top of built-in rule
- Simulation: crontab entry with reverse shell payload injected from Kali via SSH, then removed. Alert fired: "CTI T1053.005 - Scheduled Task Persistence detected (QakBot TTP)" — IRIS Case #47 auto-created

## 15.2 Cycle 2 — CVE-2026-31431 Copy Fail Linux Kernel LPE

- CTI Source: Elastic Security Labs — "Copy Fail and DirtyFrag: Linux page bugs in the wild" (published May 9, 2026). CVE listed in CISA Known Exploited Vulnerabilities catalog.
- Vulnerability: `af_alg` kernel module (loaded, confirmed via `lsmod`) enables non-root processes to open `AF_ALG` sockets (socket family 38) and use `splice()` to corrupt page cache — potential privilege escalation to root. Ubuntu-victim kernel 6.8.0-111-generic confirmed vulnerable. No patch available at time of lab.
- Detection: auditd rules deployed per Elastic guidance (`-a always,exit -F arch=b64 -S socket -k socket_syscall` and `splice, bind, unshare` variants). Wazuh agent configured to read `/var/log/audit/audit.log` as audit format log source.
- Wazuh Rule 100030 (level 14, MITRE T1068): triggers on auditd key `socket_syscall` when `a0=26` (`AF_ALG` family) from non-root UID. Rule 100031 (level 14, T1611): namespace manipulation via `unshare`. Rule 100032 (level 15, T1068): `splice()` syscall detection.

- Simulation result: ausearch confirmed syscall=41 (socket), a0=26, uid=1000 (kendo), exe=/usr/bin/python3.12, key=socket\_syscall. Alert fired: "Copy Fail CVE-2026-31431 - AF\_ALG socket primitive detected" level 14 — IRIS Case #83 auto-created by Shuffle.
- Mitigations applied: algif\_aead blocked via /etc/modprobe.d/copyfail.conf; esp4, esp6, rxrpc blocked via /etc/modprobe.d/dirtyfrag.conf; kernel.unprivileged\_usersns\_clone=0 via sysctl; drop\_caches executed. Pending: kernel update when Ubuntu releases patched linux-image for 6.8.0-112+.

## 16. Updated MITRE ATT&CK Coverage

The full lab now covers the following MITRE ATT&CK techniques across all phases and CTI cycles:

- T1595 Active Scanning — Nmap NSE reconnaissance — Rule 100012 — Reconnaissance
- T1190 Exploit Public-Facing Application — SQLi auth bypass, SQLMap — Rules 31106, 100011, 100013 — Initial Access
- T1059.007 JavaScript Execution — XSS injection — Rules 31105, 100015 — Execution
- T1053.005 Scheduled Task — QakBot-style cron persistence — Rule 100020 — Persistence
- T1068 Exploitation for Privilege Escalation — CVE-2026-31431 Copy Fail AF\_ALG — Rules 100030, 100032 — Privilege Escalation
- T1611 Escape to Host — DirtyFrag namespace manipulation — Rule 100031 — Privilege Escalation
- T1110 Brute Force — Hydra credential attack — Rule 100010 — Credential Access
- T1046 Network Service Discovery — Nmap internal scanning — Rule 100021 — Discovery
- T1078 Valid Accounts — admin endpoint access — Rule 100014 — Defense Evasion
- T1195 Supply Chain Compromise — CI/CD pipeline security scanning — Gitea Actions + Trivy/Snyk — Defensive coverage

## 17. Full Infrastructure Summary

### 17.1 Virtual Machine Inventory (Final State)

- 192.168.75.132 ubuntu-victim — Juice Shop v19.2.1, Nginx, ModSecurity WAF, Wazuh Agent, Trivy, Snyk, Gitea, Docker, auditd
- 192.168.75.139 ubuntu-server — Wazuh SIEM v4.14.5 (manager, indexer, dashboard), 9 custom detection rules
- 192.168.75.140 shuffle-soar — Shuffle SOAR (Docker), DFIR-IRIS v2.4.29 (Docker)
- 192.168.75.142 kali — Attacker platform (Nmap, Hydra, SQLMap, curl, custom scripts)
- 192.168.75.145 misp-server — MISP v2.4 (Ubuntu 22.04), Apache, MariaDB, Redis, 5 active threat intel feeds

### 17.2 Custom Wazuh Detection Rules — Complete List

- 100010 L10 — Hydra brute force detected — T1110
- 100011 L10 — SQLMap scanner detected — T1190
- 100012 L8 — Nmap NSE scanner detected — T1595

- 100013 L12 — Multiple WAF blocks same IP (10 in 60s) — T1190 — triggers Active Response iptables block
- 100014 L10 — Juice Shop admin endpoint accessed — T1078
- 100015 L10 — XSS attempt detected — T1059.007 — triggers Shuffle SOAR pipeline
- 100016 L5 — Qualys scanner activity (whitelisted) — T1595
- 100020 L12 — CTI T1053.005 Scheduled Task Persistence (QakBot TTP) — T1053.005
- 100030 L14 — CVE-2026-31431 Copy Fail AF\_ALG socket primitive — T1068 (auditd source)
- 100031 L14 — DirtyFrag namespace manipulation via unshare — T1611 (auditd source)
- 100032 L15 — splice() syscall page cache manipulation primitive — T1068 (auditd source)

## Appendix B - Next Steps and Roadmap

- LLM-based T1/T2 triage — integrate Ollama (Qwen3.5 or Mistral 7B) as a Shuffle SOAR node to automatically classify alert severity and generate analyst recommendations before IRIS case creation
- CMDB/Asset identity — lightweight JSON asset registry in Shuffle to enrich alerts with asset owner, criticality, and business context
- Additional CTI cycles — Atomic Red Team for structured TTP simulation; Sigma rule development for portable detection logic exportable to other SIEMs
- Kernel patch validation — retest CVE-2026-31431 detection after Ubuntu releases linux-image patch for 6.8.0-112+; confirm auditd rules remain effective
- MSSP model exploration — evaluate productizing this open-source stack as a managed security service for SMB clients in Costa Rica and Central America, targeting sectors with regulatory requirements (SUGEF, PCI DSS, Ley 8968)

## Appendix A - Configuration Reference

### Phase 3 - Advanced Attack Techniques

- Burp Suite manual exploitation - IDOR, broken access control, business logic flaws
- WAF bypass techniques - encoding, case variation, chunked transfer encoding
- SSRF (Server-Side Request Forgery) against internal services
- JWT token manipulation - algorithm confusion and signature bypass
- File upload vulnerabilities and web shell delivery through Juice Shop

### Phase 4 - Detection Engineering

- Wazuh active response - automatic IP blocking on attack threshold breach
- Custom decoders for structured Juice Shop JSON application logs
- SIEM dashboard - attack timeline, top attacker IPs, rule fire frequency charts
- Threat hunting queries - proactive IOC search in historical data
- Sigma rule development - portable detection rules exportable to other SIEMs

### Phase 5 - Incident Response Simulation

- Full attack chain simulation: recon to exploit to persistence to exfiltration
- IR runbook development for each attack type documented in this report
- Purple team exercise - attacker and defender working simultaneously
- Forensic artifact collection and chain of custody documentation

## Phase 6 - Compliance & Hardening

- CIS Ubuntu 24.04 Benchmark - review Wazuh SCA scan results and remediate findings
- PCI DSS controls mapping using Wazuh built-in compliance dashboard
- NIST 800-53 control implementation review
- Vulnerability scanning with OpenVAS against victim VM

# Appendix A - Configuration Reference

## A.1 Systemd Services - Victim VM

| Service             | Status           | Enabled | Purpose                                          |
|---------------------|------------------|---------|--------------------------------------------------|
| juiceshop.service   | active (running) | yes     | OWASP Juice Shop Node.js app on port 3000        |
| nginx.service       | active (running) | yes     | Reverse proxy + ModSecurity WAF on port 80       |
| wazuh-agent.service | active (running) | yes     | Security telemetry agent - forwards logs to SIEM |
| systemd-timesyncd   | active (running) | yes     | NTP time synchronization                         |

## A.2 Wazuh Manager Services - SIEM VM

| Service         | Status           | Purpose                                |
|-----------------|------------------|----------------------------------------|
| wazuh-manager   | active (running) | Core analysis and correlation engine   |
| wazuh-indexer   | active (running) | OpenSearch alert storage and indexing  |
| wazuh-dashboard | active (running) | Web interface on HTTPS port 443        |
| filebeat        | active (running) | Log forwarding from manager to indexer |

## A.3 Snapshot History - Victim VM

| Snapshot Name                              | State | Purpose                                  |
|--------------------------------------------|-------|------------------------------------------|
| Fase1-SinWAF-JuiceShop+Nginx+WazuhAgent-OK | Saved | Baseline before WAF installation         |
| Fase2-WAF-ModSecurity-CRS-Wazuh-Validado   | Saved | Full stack with WAF active and validated |

## A.4 Wazuh Agent Log Sources (ossec.conf)

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/juiceshop_access.log</location>
</localfile>
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/juiceshop_error.log</location>
</localfile>
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/auth.log</location>
</localfile>
<localfile>
  <log_format>journald</log_format>
  <location>journald</location>
</localfile>
```